

Hashing

Collision Resolution by Chaining

1 Collision Resolution by Chaining

When multiple keys hash to the same slot, a **collision** occurs.

In **chaining**, we resolve collisions by storing all elements that hash to the same slot in a linked list.

Important Concept

Each slot j of the hash table contains:

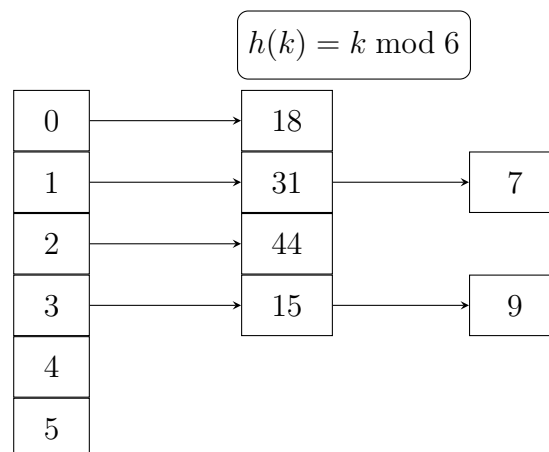
$$T[j] = \text{pointer to head of linked list}$$

If no elements hash to slot j , then:

$$T[j] = \text{NIL}$$

Thus, the hash table is an array of linked lists.

2 Hashing with Chaining



3 Dictionary Operations with Chaining

Let $h(k)$ be the hash function.

1. Insertion

Algorithm:

- Compute $j = h(x.key)$
- Insert x at the **head** of list $T[j]$

Worst-case time: $O(1)$

Key Idea

Insertion is fast because we assume the key is not already present. If duplicate checking is required, we must search first (extra cost).

2. Search

Algorithm:

- Compute $j = h(k)$
- Search linked list $T[j]$

Worst-case time: $\Theta(n)$

(Occurs when all keys hash to the same slot.)

3. Deletion

If lists are **doubly linked**:

Deletion time = $O(1)$

If lists are **singly linked**:

- Must search for predecessor
- Deletion becomes $\Theta(n)$

Important Concept

If deletion is required, use **doubly linked lists**.

4 Load Factor

Definition 1 (Load Factor). If a hash table has:

- m slots
- n stored elements

Then the load factor is:

$$\alpha = \frac{n}{m}$$

α = average number of elements per chain

- $\alpha < 1 \rightarrow$ Sparse table
- $\alpha = 1 \rightarrow$ Balanced
- $\alpha > 1 \rightarrow$ Heavy chaining

5 Worst-Case Behavior

Worst case occurs when:

All n keys hash to the same slot

Search time = $\Theta(n)$

This is no better than using a single linked list.

Important Concept

Hash tables are not designed for worst-case performance. They are optimized for **average-case performance**.

6 Simple Uniform Hashing Assumption

We assume:

Each key is equally likely to hash to any of the m slots, independently.

Simple Uniform Hashing

Let n_j denote the length of chain j (i.e., the number of keys that hash to slot j).

We compute its expected value.

For each key k_i , define the indicator variable:

$$Y_{ij} = \begin{cases} 1 & \text{if } h(k_i) = j \\ 0 & \text{otherwise} \end{cases}$$

Then the total number of elements in chain j is:

$$n_j = \sum_{i=1}^n Y_{ij}$$

Taking expectation and applying linearity:

$$E[n_j] = E \left[\sum_{i=1}^n Y_{ij} \right] = \sum_{i=1}^n E[Y_{ij}]$$

Since each key is equally likely to hash to any of the m slots,

$$\Pr(h(k_i) = j) = \frac{1}{m}$$

and because Y_{ij} is an indicator variable,

$$E[Y_{ij}] = \frac{1}{m}$$

Therefore,

$$E[n_j] = \sum_{i=1}^n \frac{1}{m} = \frac{n}{m} = \alpha$$

$$\boxed{E[n_j] = \alpha}$$

where n_j is the length of chain j .

7 Unsuccessful Search Analysis

Theorem 1. *In chaining with simple uniform hashing, an unsuccessful search takes*

$$\Theta(1 + \alpha)$$

average-case time.

Proof Idea:

- Expected chain length = α
- Examine all elements in that chain
- Add $O(1)$ time for computing $h(k)$

$$\text{Expected time} = \alpha + O(1) = \Theta(1 + \alpha)$$

8 Successful Search Analysis

Theorem 2. *In a hash table with chaining under simple uniform hashing, a successful search takes*

$$\Theta(1 + \alpha)$$

average-case time.

Proof

Assume:

- n elements stored.
- Inserted in order $x_1, x_2, \dots, x_n$.
- Insertions occur at the front.
- Search target equally likely among n elements.

Step 1: Elements Examined

$$\text{Elements examined} = 1 + \sum_{j=i+1}^n X_{ij}$$

Step 2: Indicator Variables

For keys k_i and k_j ($j > i$), define the indicator random variable

$$X_{ij} = \begin{cases} 1 & \text{if } h(k_i) = h(k_j) \\ 0 & \text{otherwise} \end{cases}$$

Thus, X_{ij} indicates whether the two keys collide.

Since X_{ij} is an indicator random variable, we use the fundamental property:

$$E[X] = \Pr(X = 1)$$

Therefore,

$$E[X_{ij}] = \Pr(h(k_i) = h(k_j))$$

Under the assumption of **simple uniform hashing**:

- Each key is equally likely to hash into any of the m slots.
- Hash values are independent.

If k_i hashes to some slot, the probability that k_j hashes to the same slot is:

$$\Pr(h(k_i) = h(k_j)) = \frac{1}{m}$$

Hence,

$$\boxed{E[X_{ij}] = \frac{1}{m}}$$

Step 3: Expected Value

$$\begin{aligned} E &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \\ &= \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m} \right) \end{aligned}$$

Step 4: Simplification

$$= \frac{1}{n} \left(n + \frac{1}{m} \cdot \frac{n(n-1)}{2} \right) = 1 + \frac{n-1}{2m}$$

Step 5: Express Using Load Factor

$$\alpha = \frac{n}{m}$$

$$E = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

Including hash computation:

$$\Theta \left(2 + \frac{\alpha}{2} - \frac{\alpha}{2n} \right) = \Theta(1 + \alpha)$$

□

9 Final Interpretation

If:

$$n = O(m)$$

then

$$\alpha = O(1)$$

$$\boxed{\text{Search runs in } O(1) \text{ average time.}}$$

Insertion: $O(1)$ worst-case

Deletion: $O(1)$ worst-case (doubly linked)

$O(1)$ average time for all dictionary operations

10 Quick Exam Summary

- Collision resolution method: **Chaining**
- Load factor: $\alpha = n/m$
- Unsuccessful search: $\Theta(1 + \alpha)$
- Successful search: $\Theta(1 + \alpha)$
- Worst case: $\Theta(n)$
- Works well when $\alpha = O(1)$

Chaining gives constant average time — if the hash function distributes keys uniformly.
